Data-Driven Optimal Shared Control of Unmanned Aerial Vehicles

The control method obtains the UAV state, represents nearby obstacles through their positions and velocities, predicts possible close encounters, constructs an avoidance direction, and sends a smoothed position and velocity reference to the low-level controller. The same mathematical framework supports pilot-commanded motion, waypoint-guided motion, moving obstacles, and cooperative operation with multiple UAVs.

Drone state and time step

Let the position and velocity of a drone at control step k be

$$\boldsymbol{p}_d[k] = \begin{bmatrix} x_d & y_d & z_d \end{bmatrix}^T, \qquad \boldsymbol{v}_d[k] = \begin{bmatrix} v_{x,d} & v_{y,d} & v_{z,d} \end{bmatrix}^T.$$

The coordinates x_d, y_d, z_d are measured in metres and the velocity components in metres per second. The superscript T denotes transpose. The subscript d identifies the drone being controlled.

The physics frequency in the code is $f_p = 240$ Hz and the control frequency is $f_c = 48$ Hz. Therefore the controller time step is

$$\boxed{\Delta t = \frac{1}{f_c} = \frac{1}{48} \simeq 0.020833 \text{ s.}}$$

Here f_c is the number of controller updates per second. The simulation time at step k is $t_k = k\Delta t$. The physics engine may integrate at a finer rate, while the high-level commands are updated once per control step.

The state vector returned by the simulator state function contains position in entries 0 to 2, orientation quaternion in entries 3 to 6, roll, pitch and yaw in entries 7 to 9, and linear velocity in entries 10 to 12. Yaw is denoted by ψ, in radians. The high-level equations below use position, velocity and yaw; the imported controller handles the lower-level attitude and motor response.

Movement command

For manual movement, each key is represented by an indicator that is one while the key is pressed and zero otherwise. The signed inputs are differences between opposite keys:

$$u_f = I_{\text{forward}} - I_{\text{back}}, \quad u_\ell = I_{\text{left}} - I_{\text{right}}, \quad u_z = I_{\text{up}} - I_{\text{down}}.$$

Each I is either 0 or 1, so each signed input is in $\{-1, 0, 1\}$. The key bindings are implementation details; the signed inputs represent forward/backward, left/right, and vertical motion.

The horizontal directions corresponding to forward and left, expressed in world coordinates, are

$$\boldsymbol{f} = \begin{bmatrix} \cos\psi \\ \sin\psi \end{bmatrix}, \qquad \boldsymbol{\ell} = \begin{bmatrix} -\sin\psi \\ \cos\psi \end{bmatrix}.$$

The angle ψ is the drone's current yaw. These vectors rotate the body-relative movement directions into the simulation's x-y frame. Thus the unscaled manual velocity direction is

$$\boldsymbol{v}_{\text{raw}} = \begin{bmatrix} u_f \cos\psi - u_\ell \sin\psi \\ u_f \sin\psi + u_\ell \cos\psi \\ u_z \end{bmatrix}.$$

The third component is vertical movement and is not rotated by yaw.

To prevent diagonal input from producing a larger speed than straight input, the code first calculates $m = \|\boldsymbol{v}_{\text{raw}}\|$. If $m > 1$, it divides the vector by m; otherwise it leaves the vector unchanged. The resulting velocity command is

$$\boxed{\boldsymbol{v}_{\text{manual}} = V_{\max}\boldsymbol{v}_{\text{raw}} \min\left(1, \frac{1}{m}\right),}$$

where the multiplier is interpreted as 1 when $m = 0$, and $V_{\max} = 0.50$ m/s is the manual speed limit. This is the same normalization implemented by the conditional statement in the code.

In the autonomous waypoint case, normal movement instead comes from the active waypoint $\boldsymbol{p}_w$. The vector from the drone to that waypoint and its length are

$$\boldsymbol{p}_g = \boldsymbol{p}_w - \boldsymbol{p}_d, \qquad d_g = \|\boldsymbol{p}_g\|.$$

Here $\boldsymbol{p}_g$ is the goal displacement and d_g is the distance to the waypoint. If $d_g > 10^{-6}$, the unit direction toward the waypoint is

$$\boldsymbol{g} = \frac{\boldsymbol{p}_g}{d_g};$$

otherwise the code uses a fixed fallback direction $(1, 0, 0)^T$ to avoid division by zero. After the one-second warm-up, the requested goal velocity is

$$\boxed{\boldsymbol{v}_g = \boldsymbol{g}\ \min(V_c,\ 0.8 d_g),}$$

where $V_c = 0.50$ m/s is the cruise-speed limit and 0.8 is the distance-proportional gain (with units of inverse seconds when used dimensionally). This slows the request as the drone approaches the waypoint. During warm-up, $\boldsymbol{v}_g = \boldsymbol{0}$. In the shared method, write $\boldsymbol{v}_b$ for the baseline velocity: it is $\boldsymbol{v}_{\text{manual}}$ for manual piloting and $\boldsymbol{v}_g$ for waypoint flight.

Obstacle position and velocity

An obstacle is represented by its current position $\boldsymbol{p}_o$, velocity $\boldsymbol{v}_o$, and effective radius r_o. For a static obstacle, $\boldsymbol{p}_o$ is constant and $\boldsymbol{v}_o = \boldsymbol{0}$. The pillar is represented as a cylinder, while the box is approximated by a sphere for the avoidance calculation. For a box half-size of 0.30 m, the code uses the effective radius

$$r_{\text{box}} = 1.8(0.30) = 0.54 \text{ m}.$$

This spherical radius is a conservative approximation used by the avoidance logic; it is not the exact distance to the box's faces.

The moving obstacles are defined analytically as functions of simulation time. For the patrolling sphere,

$$\boldsymbol{p}_o(t) = \begin{bmatrix} 1 \\ A\sin(\omega t) \\ 1 \end{bmatrix}, \qquad \boldsymbol{v}_o(t) = \frac{\mathrm{d}\boldsymbol{p}_o}{\mathrm{d}t} = \begin{bmatrix} 0 \\ A\omega\cos(\omega t) \\ 0 \end{bmatrix},$$

where $A = 2.0$ m is the oscillation amplitude and $\omega = 0.35$ rad/s is the angular frequency. Its maximum speed is $A\omega = 0.70$ m/s.

For the orbiting sphere,

$$\boldsymbol{p}_o(t) = \begin{bmatrix} c_x + r\cos(\omega t) \\ c_y + r\sin(\omega t) \\ z_0 \end{bmatrix}, \quad \boldsymbol{v}_o(t) = \begin{bmatrix} -r\omega\sin(\omega t) \\ r\omega\cos(\omega t) \\ 0 \end{bmatrix}.$$

The orbit centre is $(c_x, c_y) = (-1.5, 1.0)$ m, radius $r = 1.2$ m, angular frequency $\omega = 0.40$ rad/s, and height $z_0 = 1.2$ m. Its speed is constant in this parametrization:

$$\|\boldsymbol{v}_o\| = r\omega = 1.2(0.40) = 0.48 \text{ m/s}.$$

For the vertically moving sphere,

$$\boldsymbol{p}_o(t) = \begin{bmatrix} 2.5 \\ y_0 \\ z_0 + A\sin(\omega t) \end{bmatrix}, \qquad \boldsymbol{v}_o(t) = \begin{bmatrix} 0 \\ 0 \\ A\omega\cos(\omega t) \end{bmatrix},$$

where $z_0 = 1.2$ m, $A = 0.6$ m and $\omega = 0.80$ rad/s. Its altitude ranges from 0.6 m to 1.8 m and its maximum vertical speed is 0.48 m/s. The fixed lateral coordinate is a scenario parameter and does not change the derivation.

When two drones are simulated, the other drone is added to the obstacle list. For the drone currently being controlled, the other drone's position and velocity are used as $\boldsymbol{p}_o$ and $\boldsymbol{v}_o$, with effective radius $r_o = 0.12$ m. The controlled drone's assumed radius is $r_d = 0.10$ m. The same relative-motion equations therefore apply to fixed obstacles, moving spheres and the other drone.

Relative position and clearance

The code uses different distance calculations for the pillar and the spherical obstacles. Define the component-wise mask

$$\boldsymbol{m} = \begin{cases} (1,1,0)^T, & \text{cylindrical pillar,} \\ (1,1,1)^T, & \text{spherical obstacle or drone.} \end{cases}$$

The symbol $\odot$ denotes component-wise multiplication. The relative position from the controlled drone to an obstacle and the relative velocity are

$$\boxed{\boldsymbol{r} = (\boldsymbol{p}_o - \boldsymbol{p}_d) \odot \boldsymbol{m}, \qquad \boldsymbol{u} = (\boldsymbol{v}_o - \boldsymbol{v}_d) \odot \boldsymbol{m}.}$$

Thus $\boldsymbol{r}$ points from the drone towards the obstacle, while $\boldsymbol{u}$ describes how that separation vector changes under the current velocities. For the pillar, the vertical component is ignored, so the calculation uses horizontal separation only. For a sphere, all three dimensions are included.

The current centre distance is

$$d = \|\boldsymbol{r}\| = \sqrt{r_x^2 + r_y^2 + r_z^2}.$$

The surface-to-surface gap used by the code is

$$\boxed{g = d - r_o - r_d,}$$

where r_o is the obstacle's effective radius and $r_d = 0.10$ m is the assumed drone radius. A positive g means a gap remains between the effective shapes; $g = 0$ means they touch; and $g < 0$ means the effective shapes overlap according to this approximation.

For all obstacles in the list, the nearest current gap is

$$g_{\min} = \min_j g_j.$$

The index j identifies an obstacle. Detailed prediction is skipped for an obstacle when its current gap is greater than $R_p = 2.5$ m. The nearest-gap value is calculated before this range test, as in the code.

Time of closest approach

The avoidance calculation estimates whether the drone and an obstacle will come closer over the next $H = 2.0$ seconds. During this prediction, relative velocity is treated as constant. If τ is a future time offset, the predicted relative position is

$$\boxed{\boldsymbol{r}(\tau) = \boldsymbol{r} + \boldsymbol{u}\tau.}$$

The predicted squared centre distance is

$$D(\tau) = \|\boldsymbol{r} + \boldsymbol{u}\tau\|^2 = (\boldsymbol{r} + \boldsymbol{u}\tau)^T(\boldsymbol{r} + \boldsymbol{u}\tau).$$

Expanding the product gives

$$D(\tau) = \boldsymbol{r}^T\boldsymbol{r} + 2\tau\,\boldsymbol{r}^T\boldsymbol{u} + \tau^2\boldsymbol{u}^T\boldsymbol{u}.$$

The closest point on this constant-velocity path occurs where the derivative of $D(\tau)$ is zero:

$$\frac{\mathrm{d}D}{\mathrm{d}\tau} = 2\boldsymbol{r}^T\boldsymbol{u} + 2\tau\boldsymbol{u}^T\boldsymbol{u} = 0.$$

Rearranging,

$$\tau^* = -\frac{\boldsymbol{r}^T\boldsymbol{u}}{\boldsymbol{u}^T\boldsymbol{u}} = -\frac{\boldsymbol{r}\cdot\boldsymbol{u}}{\|\boldsymbol{u}\|^2}.$$

The dot product $\boldsymbol{r}\cdot\boldsymbol{u}$ measures whether the relative motion is closing or opening. If it is negative, the unconstrained closest point is in the future. If it is positive, the unconstrained result lies in the past, so the future closest point is now.

The implementation avoids division by a near-zero relative speed. If $\|\boldsymbol{u}\|^2 > 10^{-6}$, it clips the result to the prediction interval; otherwise it sets the prediction time to zero:

$$\boxed{\tau_{\mathrm{CPA}} = \begin{cases} \mathrm{clip}\left(-\dfrac{\boldsymbol{r}\cdot\boldsymbol{u}}{\|\boldsymbol{u}\|^2},\, 0,\, H\right), & \|\boldsymbol{u}\|^2 > 10^{-6}, \\ 0, & \text{otherwise.} \end{cases}}$$

Here τ_{CPA} is the estimated time of closest approach and $\mathrm{clip}(x, a, b)$ limits x to the interval from a to b. The predicted relative position and gap are then

$$\boldsymbol{q} = \boldsymbol{r} + \boldsymbol{u}\tau_{\mathrm{CPA}}, \qquad g_p = \|\boldsymbol{q}\| - r_o - r_d.$$

The effective gap used for threat assessment is the smaller of current and predicted clearance:

$$\boxed{g_e = \min(g, g_p).}$$

This allows avoidance to begin when an obstacle is already close or when the constant-velocity prediction estimates a closer future pass.

Threat strength and escape direction

The code starts increasing avoidance strength when the effective gap is less than $D_a = 1.5$ m. For each obstacle,

$$\boxed{\alpha = \mathrm{clip}\left(\frac{D_a - g_e}{D_a},\, 0,\, 1\right).}$$

The value α is dimensionless and lies between zero and one. If $g_e \geq D_a$, then $\alpha = 0$ and the obstacle does not contribute an avoidance direction. If the gap is between zero and D_a, the strength rises linearly as the gap decreases. If the effective gap is zero or negative, the value clips to one. A threat is present if at least one obstacle has $\alpha > 0$. In the two autonomous/manual multi-drone versions, the largest active strength is also tracked:

$$\alpha_{\max} = \max_j \alpha_j.$$

The first avoidance direction points away from the predicted closest-approach position. Since $\boldsymbol{q}$ is relative to the drone, the away vector is $-\boldsymbol{q}$. If the predicted vector is nearly zero, the current relative vector is used instead. If that is also nearly zero, the code supplies a fallback direction to avoid dividing by zero. Denote the chosen nonzero vector by $\boldsymbol{a}_0$; its unit direction is

$$\boldsymbol{a} = \frac{\boldsymbol{a}_0}{\|\boldsymbol{a}_0\|}.$$

The code then halves its vertical component, $a_z \leftarrow 0.5a_z$, to make the escape response more strongly horizontal. The vector is normalized again after combining it with a lateral direction.

A purely radial escape direction can make a drone retreat directly backwards and remain near an obstacle. The code therefore adds a sideways component. Let the horizontal travel or heading direction be $\boldsymbol{h} = (h_x, h_y, 0)^T$, normalized in the horizontal plane. The vector to its left is

$$\boldsymbol{\ell}_h = (-h_y, h_x, 0)^T.$$

The signed lateral offset of the obstacle is

$$\delta = \boldsymbol{r}^T \boldsymbol{\ell}_h.$$

If $\delta > 0.05$ m, the obstacle is on the left and the selected passing direction is to the right, $-\boldsymbol{\ell}_h$. If $\delta < -0.05$ m, the obstacle is on the right and the selected direction is to the left, $+\boldsymbol{\ell}_h$. If $|\delta| \leq 0.05$ m, the obstacle is treated as approximately ahead and the code chooses the right-hand side. This lateral choice is stored while that obstacle remains active, preventing repeated switching between sides.

For these committed-side versions, the unit direction for obstacle j is

$$\boxed{\boldsymbol{d}_j = \frac{\boldsymbol{a}_j + 0.9\boldsymbol{s}_j}{\|\boldsymbol{a}_j + 0.9\boldsymbol{s}_j\|},}$$

where $\boldsymbol{a}_j$ is the away direction and $\boldsymbol{s}_j$ is the stored unit lateral direction. The coefficient 0.9 sets the relative contribution of the lateral motion. In the single-drone version, the side is instead selected from a horizontal tangent vector

$$\boldsymbol{t} = (-a_y, a_x, 0)^T,$$

with sign chosen from the current velocity, and the direction is

$$\boldsymbol{d}_j = \frac{\boldsymbol{a}_j + 0.5 s_j \boldsymbol{t}}{\|\boldsymbol{a}_j + 0.5 s_j \boldsymbol{t}\|}, \qquad s_j = \begin{cases} 1, & \boldsymbol{t}^T \boldsymbol{v}_d \geq 0, \\ -1, & \boldsymbol{t}^T \boldsymbol{v}_d < 0. \end{cases}$$

Both expressions implement the same principle: combine a direction away from the predicted close pass with a horizontal sideways component. The exact side-selection rule is implementation-dependent, but the weighted avoidance calculation that follows is common.

For multiple obstacles, the individual directions are weighted by their strengths and summed:

$$\boxed{\boldsymbol{e} = \sum_j \alpha_j \boldsymbol{d}_j.}$$

Here $\boldsymbol{e}$ is the aggregate escape vector. Obstacles with zero strength contribute nothing. Since the directions are summed, $\|\boldsymbol{e}\|$ may be larger than one when several obstacles push in similar directions, or small when directions oppose one another. The vector is therefore not itself a physical velocity; it is an avoidance direction and intensity used to construct the next velocity request.

Combining normal movement and avoidance

When the pilot is controlling the drone, the baseline velocity is the normalized manual command. When the drone is navigating autonomously, the baseline velocity is the waypoint velocity. The baseline velocity is combined with the avoidance contribution according to the active control mode, while the threat and escape calculations remain the same.

For a manually controlled drone in AUTO mode, the requested velocity points along the aggregate escape vector:

$$\boldsymbol{v}_{\text{des}} = \begin{cases} V_e \dfrac{\boldsymbol{e}}{\|\boldsymbol{e}\|}, & g_{\min} < D_e, \\ V_a \min(1, \|\boldsymbol{e}\|) \dfrac{\boldsymbol{e}}{\|\boldsymbol{e}\|}, & \text{otherwise}, \end{cases}$$

provided that a threat is active and $\|\boldsymbol{e}\| > 10^{-6}$. Here $D_e = 0.65$ m is the emergency gap, $V_a = 0.60$ m/s is the normal avoidance speed, and $V_e = 0.90$ m/s is the emergency speed. If AUTO is active but no usable escape vector exists, the desired velocity is zero, requesting the drone to brake or hover. In MANUAL mode the desired velocity is the baseline manual command, except when emergency avoidance overrides it.

For autonomous waypoint flight, the goal velocity is reduced as threat strength increases and the escape vector is added:

$$\boxed{\boldsymbol{v}_{\text{des}} = (1 - 0.7\alpha_{\max})\boldsymbol{v}_g + V_a \boldsymbol{e}.}$$

The coefficient 0.7 controls how strongly the goal-directed term is reduced. The result is limited to V_a in normal avoidance and V_e during an emergency. If no threat exists, the desired velocity is simply $\boldsymbol{v}_g$. This is the shared trade-off: continue towards the goal when the route is clear, but give increasing influence to the avoidance vector when a close pass is predicted.

The emergency flag is based on the nearest current gap:

$$\boxed{\text{emergency} = (g_{\min} < D_e).}$$

It is not directly based on g_e. Consequently, a predicted future conflict may activate avoidance before the current gap becomes small enough to trigger emergency mode.

Limiting the change in commanded velocity

A desired velocity can change abruptly when keys are pressed or when an obstacle enters the avoidance range. The code limits the change in the commanded velocity. Let $\boldsymbol{v}_c[k]$ be the current smoothed command and $\boldsymbol{v}_{\text{des}}[k]$ the new request. Their difference is

$$\Delta\boldsymbol{v} = \boldsymbol{v}_{\text{des}}[k] - \boldsymbol{v}_c[k], \qquad d_v = \|\Delta\boldsymbol{v}\| .$$

For a selected acceleration limit a_{lim}, the maximum change allowed during one control step is

$$\Delta v_{\max} = a_{\text{lim}} \Delta t.$$

The next command is

$$\boxed{\boldsymbol{v}_c[k+1] = \begin{cases} \boldsymbol{v}_c[k] + \Delta\boldsymbol{v}\dfrac{\Delta v_{\max}}{d_v}, & d_v > \Delta v_{\max}, \\ \boldsymbol{v}_{\text{des}}[k], & d_v \le \Delta v_{\max}. \end{cases}}$$

If the difference is too large, the code scales the whole difference vector, preserving its direction while limiting its length. Therefore,

$$\|\boldsymbol{v}_c[k+1] - \boldsymbol{v}_c[k]\| \le a_{\lim}\Delta t.$$

The normal acceleration limit is $a_{\lim} = 1.0$ m/s^2, while emergency avoidance uses 2.2 m/s^2. At 48 Hz, these correspond to maximum command changes of approximately 0.02083 m/s and 0.04583 m/s per update. This is a bound on the commanded velocity change, not a proof that the physical drone's acceleration is exactly the same.

Position reference and leash

The smoothed velocity is integrated to move the position target:

$$\boxed{\boldsymbol{p}_T^{\text{raw}}[k+1] = \boldsymbol{p}_T[k] + \boldsymbol{v}_c[k+1]\Delta t.}$$

Here $\boldsymbol{p}_T$ is the target position given to the controller, not the measured drone position. The expression is forward Euler integration: velocity multiplied by the time step gives the target displacement during one update.

The target is constrained to the arena:

$$x_T \in [-X_{\lim}, X_{\lim}], \quad y_T \in [-X_{\lim}, X_{\lim}], \quad z_T \in [Z_{\min}, Z_{\max}],$$

where $X_{\lim} = 4.5$ m, $Z_{\min} = 0.35$ m and $Z_{\max} = 2.5$ m. The code clips each coordinate independently to these intervals.

The target is then kept within a distance $L = 0.30$ m of the actual drone position. Let

$$\boldsymbol{e}_T = \boldsymbol{p}_T^{\text{raw}} - \boldsymbol{p}_d, \qquad e_T = \|\boldsymbol{e}_T\|.$$

The final target is

$$\boxed{\boldsymbol{p}_T = \begin{cases} \boldsymbol{p}_d + \boldsymbol{e}_T\dfrac{L}{e_T}, & e_T > L, \\ \boldsymbol{p}_T^{\text{raw}}, & e_T \le L. \end{cases}}$$

The vector from the actual position to the target is shortened only if its length exceeds L; its direction is retained. This leash prevents the target from moving far ahead of a drone that cannot follow the command immediately. When automatic control hands back to the pilot, the target position is reset to the measured position so the next reference begins from the drone's current location.

Yaw reference

Yaw handling is separate from the translational avoidance calculation. For manual flight, the target yaw may be held at a fixed reference. In autonomous waypoint flight, the target yaw

follows the horizontal direction of travel when the commanded horizontal speed is sufficiently large:

$$\psi_{\text{want}} = \text{atan2}(v_{c,y}, v_{c,x}).$$

The function $\text{atan2}(y, x)$ returns the angle of a two-dimensional vector with the correct quadrant. The yaw error is wrapped to the interval $[-\pi, \pi)$:

$$e_\psi = \text{wrap}(\psi_{\text{want}} - \psi_T), \qquad \text{wrap}(a) = (a + \pi) \bmod 2\pi - \pi.$$

The target yaw is updated by a limited step,

$$\Delta\psi = \text{clip}(e_\psi, -\omega_\psi \Delta t, \omega_\psi \Delta t), \qquad \psi_T[k+1] = \text{wrap}(\psi_T[k] + \Delta\psi),$$

where $\omega_\psi = 0.8$ rad/s is the yaw-reference rate limit. For pilot-controlled yaw, the target yaw changes according to the left/right yaw input $u_\psi \in \{-1, 0, 1\}$:

$$\psi_T[k+1] = \text{wrap}\big(\psi_T[k] + u_\psi \omega_\psi \Delta t\big).$$

The yaw target is retained during automatic avoidance in that version.

Waypoint and automatic-control timing

In waypoint flight, the current waypoint index is advanced when either the drone comes within $R_{\text{arrive}} = 0.6$ m of the waypoint or the waypoint timer exceeds $T_{\text{waypoint}} = 12$ s:

$$d_g < R_{\text{arrive}} \quad \text{or} \quad \tau_w > T_{\text{waypoint}}.$$

The index is updated cyclically, $j \leftarrow (j+1) \bmod N_w$, where N_w is the number of waypoints in that mission. The timer is reset after switching. The timeout prevents a drone from remaining on an unreachable waypoint indefinitely; it does not mean that the waypoint was physically reached.

For manual/automatic takeover, each controlled drone has a mode $m \in \{\text{MANUAL}, \text{AUTO}\}$. AUTO can begin when a threat exists and either the gap is in the emergency range or the pilot has released the relevant keys and the grace period has expired. The emergency test is $g_{\min} < D_e$. The grace timer decreases by the control time step:

$$\tau_g[k+1] = \max(0, \tau_g[k] - \Delta t).$$

After handover to the pilot, the grace period is set to 1.5 s to prevent a non-emergency threat from immediately triggering AUTO again.

While AUTO is active, its timer increases by Δt. The clear-path timer resets to zero whenever a threat remains and increases by Δt while the path is clear. Control can return to MANUAL after the path has remained clear for $T_{\text{release}} = 0.8$ s, when the pilot presses a key outside an emergency, or when AUTO reaches $T_{\text{auto}} = 3.0$ s outside an emergency. The emergency condition prevents normal pilot override and timeout release. For multiple UAVs, these variables are maintained independently for each vehicle, so one vehicle may be in AUTO while another remains in MANUAL.

Low-level control connection

The position target $\boldsymbol{p}_T$, target yaw ψ_T, and smoothed velocity reference $\boldsymbol{v}_c$ are passed to the imported `DSLPIDControl` object. The physical drone position and velocity are fed back from the simulator. At a general level, the tracking errors can be written as

$$\boldsymbol{e}_p = \boldsymbol{p}_T - \boldsymbol{p}_d, \qquad \boldsymbol{e}_v = \boldsymbol{v}_c - \boldsymbol{v}_d,$$

where $\boldsymbol{e}_p$ is position error and $\boldsymbol{e}_v$ is velocity error. A generic PID law is often written

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau)\ \mathrm{d}\tau + K_D \frac{\mathrm{d}e(t)}{\mathrm{d}t},$$

where $e(t)$ is a tracking error, K_P, K_I, and K_D are proportional, integral, and derivative gains, and $u(t)$ is a controller output. This formula explains the general PID principle; the available control code does not expose the imported controller's internal gains or complete control equations, so those cannot be determined from the high-level control logic alone.

The controller returns four motor RPM commands, which are passed to the simulator. The simulator then updates the physical state. The avoidance equations therefore generate a reference trajectory; they do not directly calculate the four motor speeds.

Numerical checks

For a simple closest-approach example, let

$$\boldsymbol{r} = (2, 0, 0)^T \text{ m}, \qquad \boldsymbol{u} = (-1, 0, 0)^T \text{ m/s}.$$

The obstacle is two metres ahead and the relative motion is closing. The CPA time is

$$\tau_{\mathrm{CPA}} = -\frac{(2)(-1)}{(-1)^2} = 2 \text{ s}.$$

For $H = 2$ s, the clipped time is also two seconds, and

$$\boldsymbol{q} = (2, 0, 0)^T + (-1, 0, 0)^T(2) = (0, 0, 0)^T.$$

The predicted centre distance is zero, so $g_p = -r_o - r_d$ and the threat strength clips to one.

For the threat-strength calculation, if $g_e = 0.90$ m and $D_a = 1.5$ m,

$$\alpha = \mathrm{clip}\left(\frac{1.5 - 0.90}{1.5}, 0, 1\right) = 0.4.$$

This obstacle contributes $0.4\boldsymbol{d}$ to the aggregate escape vector, where $\boldsymbol{d}$ is its unit avoidance direction.

For the slew limiter, suppose the current command is zero and the desired command is $(0.5, 0, 0)^T$ m/s. With $a_{\mathrm{lim}} = 1.0$ m/s^2 and $\Delta t = 1/48$ s,

$$\Delta v_{\max} = 1.0/48 = 0.02083 \text{ m/s}.$$

The next command is therefore approximately $(0.02083, 0, 0)^T$ m/s, rather than an immediate jump to 0.5 m/s.

Meaning and limits of the derivation

The equations describe the high-level control method: movement-command generation, obstacle kinematics, relative geometry, closest-approach prediction, threat weighting, escape-direction construction, velocity smoothing, bounded position references, yaw handling, and control-mode timing. The same obstacle-clearance and closest-approach calculations apply to static objects, analytically moving obstacles, and other UAVs.

The method is an approximation rather than a formal collision-avoidance guarantee. The CPA calculation assumes constant relative velocity over the prediction horizon even though the drone

can accelerate and the moving obstacles follow curved or oscillating paths. The box is represented by an effective sphere and the pillar is treated using horizontal distance only. The two drones make local avoidance decisions rather than solving one joint trajectory-optimization problem. Finally, camera images are used for visualization; obstacle positions are obtained directly from the simulation and are not inferred from camera pixels.